

M/M/1 待ち行列のシミュレーション

今村隼人

2026-06-08

1. 目的

M/M/1 待ち行列について、発生イベント数と初期に無視するイベント数を変えながらシミュレーションし、平均系内客数を理論値と比較する。サービス率は $\mu = 5$ に固定し、到着率は $\lambda = 3, 4, 4.5, 4.9, 4.95, 5.1$ とした。

2. モデルと理論値

M/M/1 モデルでは、到着間隔がパラメータ λ の指数分布、サービス時間がパラメータ μ の指数分布に従う。時刻 t の系内客数を $N(t)$ とすると、 $\lambda < \mu$ のとき定常分布は

$$P(N = n) = (1 - \rho)\rho^n, \quad \rho = \frac{\lambda}{\mu}$$

である。したがって平均系内客数の理論値は

$$E[N] = \frac{\rho}{1 - \rho} = \frac{\lambda}{\mu - \lambda}$$

となる。 $\lambda \geq \mu$ では安定条件を満たさないため、定常分布は存在せず、平均系内客数は長時間では発散すると考えられる。

3. シミュレーション方法

第8回ノートブックのイベント駆動型シミュレーションを使い、系内客数が0のときは次の到着だけを発生させ、系内客数が1以上のときは到着またはサービス完了の早い方を次イベントとして発生させた。平均系内客数は、初期イベントを無視したあと、各状態で過ごした時間を重みとして

$$\bar{N} = \frac{\sum_i N_i \Delta t_i}{\sum_i \Delta t_i}$$

により計算した。乱数 seed を固定して、再実行しても同じ結果になるようにした。

実験設定は次の6通りである。

発生イベント数	無視した最初のイベント数
110	10
1100	100
11000	1000
110000	10000
1100000	100000
11000000	1000000

4. 平均系内客数

λ	発生イベント数	無視イベント数	シミュレーション	理論値	差
3	110	10	1.2405	1.5000	0.2595
3	1100	100	2.0639	1.5000	0.5639
3	11000	1000	1.4339	1.5000	0.0661
3	110000	10000	1.4855	1.5000	0.0145
3	1100000	100000	1.4972	1.5000	0.0028

λ	発生イベント数	無視イベント数	シミュレーション	理論値	差
3	11000000	1000000	1.5019	1.5000	0.0019
4	110	10	2.2323	4.0000	1.7677
4	1100	100	2.8391	4.0000	1.1609
4	11000	1000	4.1658	4.0000	0.1658
4	110000	10000	3.9374	4.0000	0.0626
4	1100000	100000	3.9784	4.0000	0.0216
4	11000000	1000000	3.9852	4.0000	0.0148
4.5	110	10	3.3569	9.0000	5.6431
4.5	1100	100	16.5422	9.0000	7.5422
4.5	11000	1000	10.4398	9.0000	1.4398
4.5	110000	10000	9.2326	9.0000	0.2326
4.5	1100000	100000	8.9029	9.0000	0.0971
4.5	11000000	1000000	8.9934	9.0000	0.0066
4.9	110	10	7.5640	49.0000	41.4360
4.9	1100	100	14.1234	49.0000	34.8766
4.9	11000	1000	41.6564	49.0000	7.3436
4.9	110000	10000	44.9363	49.0000	4.0637
4.9	1100000	100000	50.5348	49.0000	1.5348
4.9	11000000	1000000	49.4291	49.0000	0.4291
4.95	110	10	1.4438	99.0000	97.5562
4.95	1100	100	10.6584	99.0000	88.3416
4.95	11000	1000	58.9966	99.0000	40.0034
4.95	110000	10000	57.7702	99.0000	41.2298
4.95	1100000	100000	92.2279	99.0000	6.7721
4.95	11000000	1000000	105.5056	99.0000	6.5056
5.1	110	10	2.0531	∞	-
5.1	1100	100	17.1925	∞	-
5.1	11000	1000	104.1951	∞	-
5.1	110000	10000	542.4190	∞	-
5.1	1100000	100000	5197.8378	∞	-
5.1	11000000	1000000	56658.0836	∞	-

イベント数が少ない場合は、初期状態や偶然の偏りの影響が大きく、理論値から離れることがある。一方、イベント数を増やすと、安定条件を満たす $\lambda < \mu$ の範囲ではおおむね理論値へ近づいた。

5. サンプルパス

以下は、各 λ について、最大イベント数 11000000、無視イベント数 1000000 の条件で、初期イベントを除いたあとの一部時間区間を描いたサンプルパスである。

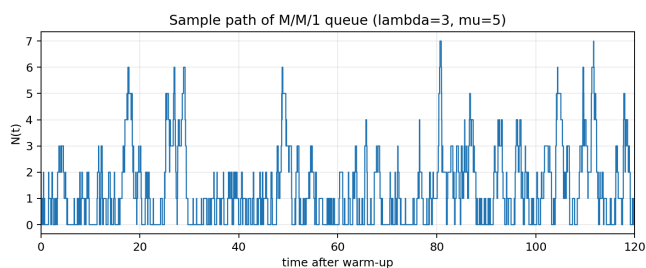


Figure 1: $\lambda = 3$

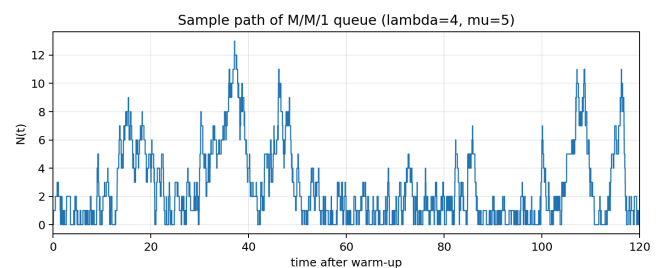


Figure 2: $\lambda = 4$

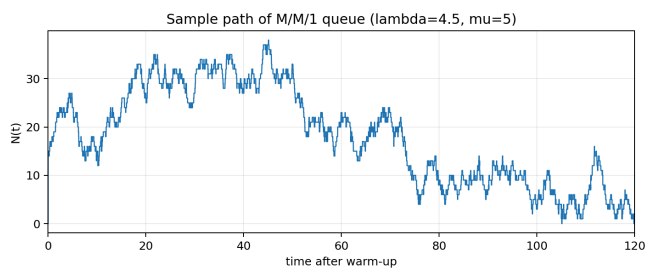


Figure 3: $\lambda = 4.5$

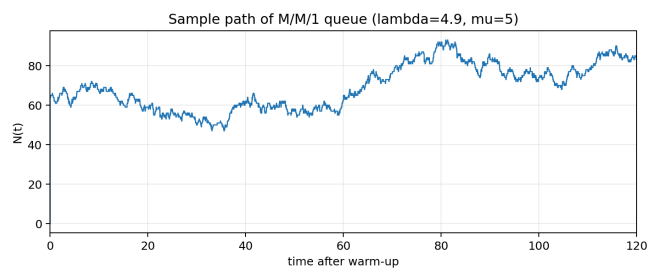


Figure 4: $\lambda = 4.9$

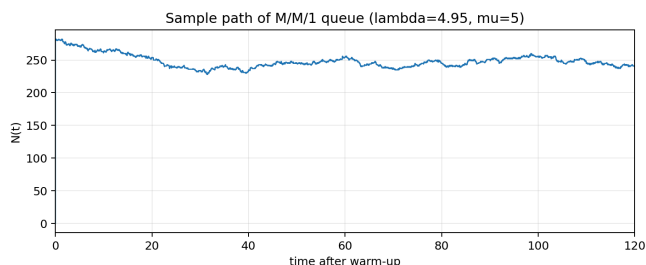


Figure 5: $\lambda = 4.95$

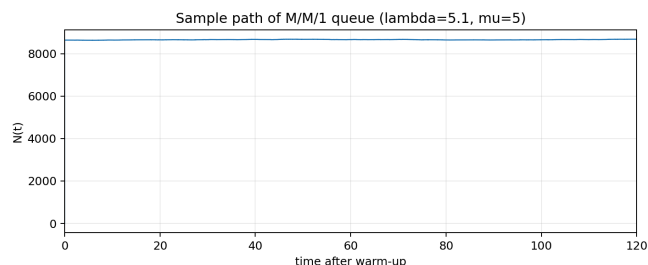


Figure 6: $\lambda = 5.1$

λ が大きくなるほど、サンプルパスの中心が上に移動し、大きな混雑状態が長く続きやすくなった。特に $\lambda = 4.95$ は $\mu = 5$ に非常に近いため、安定条件は満たしているが、混雑が解消するまでの時間が長い。 $\lambda = 5.1$ は $\lambda > \mu$ であり、サンプルパスでも系内客数が減らずに増え続ける傾向が見える。

6. 系内客数の分布

以下は、最大イベント数の条件で得た時間重み付きの系内客数分布である。 $\lambda < \mu$ の場合は幾何分布の理論値も重ねて示した。

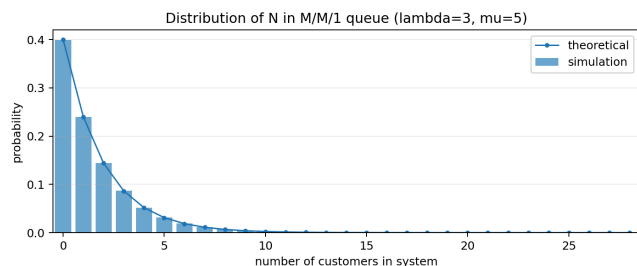


Figure 7: $\lambda = 3$

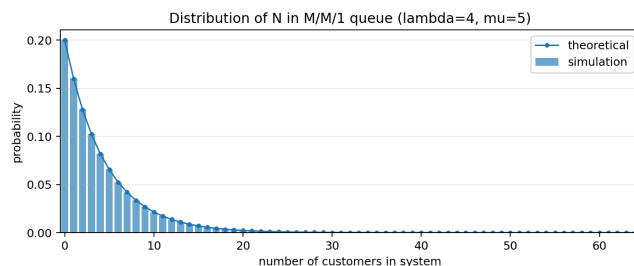


Figure 8: $\lambda = 4$

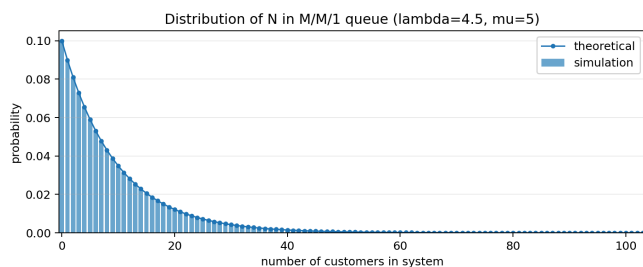


Figure 9: $\lambda = 4.5$

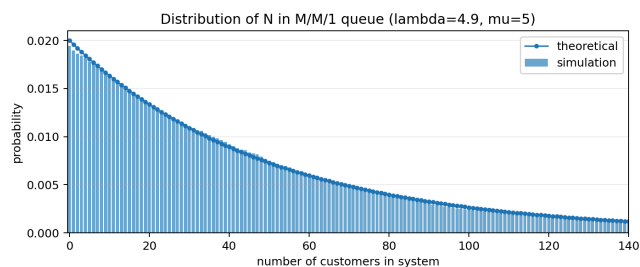


Figure 10: $\lambda = 4.9$

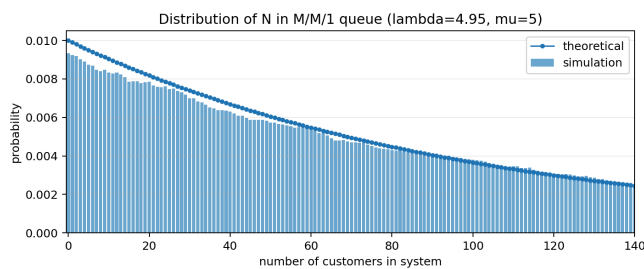


Figure 11: $\lambda = 4.95$

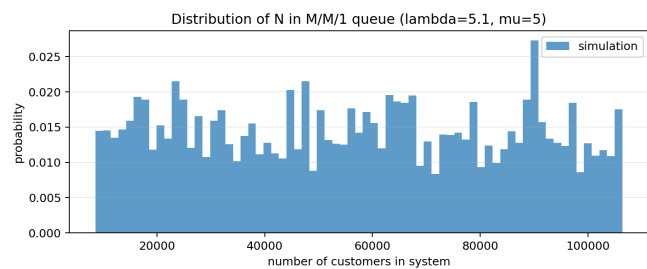


Figure 12: $\lambda = 5.1$

$\lambda = 3$ や $\lambda = 4$ では、分布が比較的低い系内客数に集中しており、シミュレーション分布も理論分布に近い。 $\lambda = 4.9$ や $\lambda = 4.95$ では分布の裾が長くなり、高い系内客数も無視できなくなる。このため、短いシミュレーションでは大きな混雑を十分に観測できず、平均値が理論値から外れやすい。

7. 考察

平均系内客数の結果を見ると、安定条件を満たす $\lambda < 5$ では、イベント数を増やすほどシミュレーション値が理論値に近づいた。例えば $\lambda = 3$ では最大設定で 1.5019 となり、理論値 1.5 とほぼ一致した。 $\lambda = 4.5$ でも最大設定では 8.9934 であり、理論値 9.0 に近い。

一方、 λ が μ に近づくと収束は遅くなる。 $\lambda = 4.9$ の理論値は 49、 $\lambda = 4.95$ の理論値は 99 であり、平均系内客数そのものが大きい。サンプルパスでも長い混雑期間と短い空き期間が交互に現れ、限られたイベント数では観測された混雑の長さに平均値が強く左右された。 $\lambda = 4.95$ では 11000000 イベントでも 105.5056 となり、理論値に近いがまだ数単位の差が残った。

$\lambda = 5.1$ は $\lambda > \mu$ なので、平均到着率が平均サービス率を上回る。この場合、定常状態は存在しないため理論値は有限にならない。実際、イベント数を増やすと平均系内客数は 2.0531 から 56658.0836 まで増加し、システムが不安定であることが確認できた。

8. 社会での応用例と感想

M/M/1 モデルは、1 台の ATM、1 人の窓口担当者、1 つの受付カウンター、1 本の通信回線など、単一のサーバにランダムに仕事や客が到着する場面の近似に使える。例えば、大学の窓口で証明書発行を受け付ける場合、到着率がサービス率に近づくと待ち人数が急増する。このモデルを使えば、担当者を増やすべき時間帯や、予約制により到着率を平準化する効果を考えられる。

今回の演習では、 λ が μ に近づくだけで平均系内客数が急激に増えることを、理論式だけでなくサンプルパスでも確認できた。短いシミュレーションでは理論値とかなりずれる場合があり、特に混雑しやすい条件ほど長い観測が必要になることも分かった。待ち行列は単純なモデルでも、現実の混雑対策を考えるうえでかなり直感的な道具になると感じた。

9. ソースコード

以下の Python コードで表と図を生成した。

```
1 import csv
2 import math
3 from pathlib import Path
4
5 import matplotlib.pyplot as plt
6 import numpy as np
7
8 MU = 5.0
9 LAMBDA = [3.0, 4.0, 4.5, 4.9, 4.95, 5.1]
10 SETTINGS = [
11     (110, 10),
12     (1100, 100),
13     (11000, 1000),
14     (110000, 10000),
15     (1100000, 100000),
16     (11000000, 1000000),
17 ]
18 BASE_SEED = 20260608
19 OUT_DIR = Path(__file__).resolve().parent
20 FIG_DIR = OUT_DIR / "figures"
21
22
23 def theoretical_mean(lam: float, mu: float) -> float:
24     if lam >= mu:
25         return math.inf
26     return lam / (mu - lam)
27
28
29 def simulate_one_lambda(
30     lam: float,
31     mu: float,
32     settings: list[tuple[int, int]],
33     seed: int,
34     path_time_limit: float = 120.0,
35 ) -> tuple[
36     list[dict[str, float | int | str]],
37     list[tuple[float, int]],
38     dict[int, float],
39 ]:
40     """Run one M/M/1 sample path and collect all event-count settings.
41
42     This is the same event-driven idea as the lecture notebook:
43     while the system has customers, the next event is the earlier of an
44     arrival clock and a service-completion clock. Equivalently, the next
45     interval has rate lambda + mu and is an arrival with probability
46     lambda / (lambda + mu). At state 0, only arrivals can happen.
47     """
48
49     rng = np.random.default_rng(seed)
50     max_events = max(total for total, _ in settings)
51     max_setting_total, max_setting_ignore = settings[-1]
52
53     numerators = [0.0 for _ in settings]
54     included_times = [0.0 for _ in settings]
55     path_points: list[tuple[float, int]] = [(0.0, 0)]
56     state_durations: dict[int, float] = {}
57
58     n_in_system = 0
59     valid_time_for_path = 0.0
60
61     for event in range(1, max_events + 1):
62         before = n_in_system
63
64         if before == 0:
65             interval = float(rng.exponential(1.0 / lam))
66             n_in_system = 1
67         else:
68             interval = float(rng.exponential(1.0 / (lam + mu)))
69             if rng.random() < lam / (lam + mu):
70                 n_in_system += 1
71             else:
72                 n_in_system -= 1
73
74         for idx, (total_events, ignored_events) in enumerate(settings):
```

```

75         if ignored_events < event <= total_events:
76             numerators[idx] += interval * before
77             included_times[idx] += interval
78
79         if max_setting_ignore < event <= max_setting_total:
80             state_durations[before] = state_durations.get(before, 0.0) + interval
81             if valid_time_for_path <= path_time_limit:
82                 path_points.append((valid_time_for_path, before))
83                 valid_time_for_path += interval
84                 path_points.append((valid_time_for_path, before))
85
86     rows: list[dict[str, float | int | str]] = []
87     theory = theoretical_mean(lam, mu)
88     for idx, (total_events, ignored_events) in enumerate(settings):
89         sim_mean = numerators[idx] / included_times[idx]
90         absolute_error = abs(sim_mean - theory) if math.isfinite(theory) else math.nan
91         rows.append(
92             {
93                 "lambda": lam,
94                 "mu": mu,
95                 "total_events": total_events,
96                 "ignored_events": ignored_events,
97                 "simulated_mean": sim_mean,
98                 "theoretical_mean": theory,
99                 "absolute_error": absolute_error,
100             }
101         )
102
103     return rows, path_points, state_durations
104
105
106 def format_lambda(lam: float) -> str:
107     return str(lam).replace(".", "p")
108
109
110 def plot_sample_path(lam: float, points: list[tuple[float, int]]) -> None:
111     xs = [x for x, _ in points]
112     ys = [y for _, y in points]
113     plt.figure(figsize=(8.0, 3.4))
114     plt.plot(xs, ys, linewidth=1.0)
115     plt.xlabel("time after warm-up")
116     plt.ylabel("N(t)")
117     plt.title(f"Sample path of M/M/1 queue (lambda={lam:g}, mu={MU:g})")
118     plt.xlim(0, min(120, max(xs) if xs else 120))
119     plt.grid(alpha=0.25)
120     plt.tight_layout()
121     plt.savefig(FIG_DIR / f"sample_path_lambda_{format_lambda(lam)}.png", dpi=180)
122     plt.close()
123
124
125 def plot_distribution(lam: float, durations: dict[int, float]) -> None:
126     total_time = sum(durations.values())
127     max_state = max(durations)
128     states = np.arange(max_state + 1)
129     probs = np.array([durations.get(int(state), 0.0) / total_time for state in states])
130
131     plt.figure(figsize=(8.0, 3.4))
132     if lam < MU:
133         plt.bar(states, probs, width=0.8, alpha=0.65, label="simulation")
134         rho = lam / MU
135         theory = (1.0 - rho) * (rho**states)
136         plt.plot(
137             states,
138             theory,
139             marker="o",
140             markersize=3,
141             linewidth=1.2,
142             label="theoretical",
143         )
144         plt.xlim(-0.5, min(max_state + 0.5, 140))
145     else:
146         observed_states = np.array(sorted(durations))
147         observed_weights = np.array(
148             [durations[int(state)] / total_time for state in observed_states]
149         )
150         bins = np.linspace(observed_states.min(), observed_states.max(), 70)
151         plt.hist(
152             observed_states,
153             bins=bins,

```

```

154         weights=observed_weights,
155         alpha=0.70,
156         label="simulation",
157     )
158     plt.xlabel("number of customers in system")
159     plt.ylabel("probability")
160     plt.title(f"Distribution of N in M/M/1 queue (lambda={lam:g}, mu={MU:g}")
161     plt.grid(axis="y", alpha=0.25)
162     plt.legend()
163     plt.tight_layout()
164     plt.savefig(FIG_DIR / f"distribution_lambda_{format_lambda(lam)}.png", dpi=180)
165     plt.close()
166
167
168 def write_results(rows: list[dict[str, float | int | str]]) -> None:
169     with (OUT_DIR / "mml_results.csv").open("w", newline="", encoding="utf-8") as f:
170         writer = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
171         writer.writeheader()
172         writer.writerows(rows)
173
174
175 def write_typst_table(rows: list[dict[str, float | int | str]]) -> None:
176     lines = [
177         "#table(",
178         "  columns: (0.7fr, 1fr, 1fr, 1.2fr, 1.2fr, 1.1fr),",
179         "  inset: 4pt,",
180         "  stroke: 0.45pt,",
181         "  table.header(",
182         "    [\$lambda\$], [生イベント数], [無視イベント数], "
183         "    [シミュレーション], [理論値], [差]),",
184     ]
185     for row in rows:
186         theory = row["theoretical_mean"]
187         theory_text = (
188             "\$infinity\$" if math.isinf(float(theory)) else f"{float(theory):.4f}"
189         )
190         err = row["absolute_error"]
191         err_text = "-" if math.isnan(float(err)) else f"{float(err):.4f}"
192         lines.append(
193             f"    [{{float(row['lambda']):g}},",
194             f"    [{{int(row['total_events'])}}],",
195             f"    [{{int(row['ignored_events'])}}],",
196             f"    [{{float(row['simulated_mean']):.4f}},",
197             f"    [{{theory_text}}],",
198             f"    [{{err_text}}],",
199         )
200     lines.append(")")
201     (OUT_DIR / "result_table.typ").write_text("\n".join(lines) + "\n", encoding="utf-8")
202
203
204 def main() -> None:
205     FIG_DIR.mkdir(parents=True, exist_ok=True)
206     all_rows: list[dict[str, float | int | str]] = []
207
208     for index, lam in enumerate(LAMBDA_S):
209         rows, path_points, durations = simulate_one_lambda(
210             lam=lam,
211             mu=MU,
212             settings=SETTINGS,
213             seed=BASE_SEED + index,
214         )
215         all_rows.extend(rows)
216         plot_sample_path(lam, path_points)
217         plot_distribution(lam, durations)
218
219     write_results(all_rows)
220     write_typst_table(all_rows)
221
222
223 if __name__ == "__main__":
224     main()

```